

Talento Digital Reinvéntate - SENCE
ESPECIALIDAD ARQUITECTO CLOUD

Diseño e Implementación de una ”Infraestructura Viva” en AWS Cloud

Autor: Jairo Morales

Fecha: 8 de julio de 2026

Resumen

Este documento presenta el desarrollo del Portafolio 4, un proyecto práctico que forma parte del sistema de evaluación continua del bootcamp *Especialidad Arquitecto Cloud* y que tiene como objetivo demostrar las habilidades adquiridas durante su desarrollo. En este caso de estudio, se diseña e implementa la infraestructura en la nube para una empresa llamada ACME. Todo el código de automatización, scripts de despliegue y archivos de configuración utilizados se encuentran documentados y disponibles en el repositorio de GitHub:

`https://github.com/donmatatan/aws-cloud-architecture-project`.

Índice

1. Justificación de servicios	1
1.1. Lección 1 – Almacenamiento	1
1.1.1. Políticas de Retención y Respaldo Automático	1
1.2. Lección 2 – Bases de Datos Relacionales	2
1.2.1. Configuración AWS RDS con PostgreSQL	2
1.2.2. Pruebas en SQLiteOnline	2
1.3. Lección 3 – Base de Datos NoSQL	4
1.3.1. Estrategia de Respaldo para DynamoDB	5
1.3.2. Integración funcional con una aplicación	5
1.4. Lección 4 – Servicios de Cómputo	7
1.4.1. Prueba de Escalabilidad bajo Carga	7
1.5. Lección 5 – Servicios de Red en la Nube	9
1.6. Lección 6 – Notificación y Mensajería	10
1.6.1. Política de reintentos	10
1.7. Lección 7 – Servicios Simples de Alojamiento Web y Contenidos	10
1.7.1. Configuraciones de seguridad para el servicio web	11
1.8. Lección 8 – Monitoreo y Correlación de Incidentes	12
1.8.1. Acción Automática (Notificación a SNS)	13
2. Diagrama de Arquitectura	15
3. Conclusión	17

1. Justificación de servicios

Esta sección presenta la justificación técnica de la infraestructura en la nube propuesta para la empresa ACME.

Toda la infraestructura descrita ha sido automatizada a través de un script ejecutable en Bash utilizando comandos de AWS CLI. El código fuente completo, junto con los archivos de configuración y los registros de las pruebas de carga, está disponible para su revisión y reproducción en el siguiente repositorio público de GitHub: <https://github.com/donmatatan/aws-cloud-architecture-project>. En dicho repositorio se localiza el script principal `despliegue.sh`, el cual orquesta la creación y validación progresiva de cada uno de los componentes analizados a continuación.

1.1. Lección 1 – Almacenamiento

Para cumplir con los requerimientos de disponibilidad, optimización de costos y retención de información histórica, se diseñó una solución de almacenamiento basada en dos buckets independientes de Amazon S3:

- **Bucket `acme-datos-frecuentes`:** Actúa como el punto de entrada operativo para todos los archivos de la empresa (para este caso logs). Este bucket no es exclusivo para datos frecuentes, sino que maneja un esquema dinámico de almacenamiento: los archivos nuevos ingresan con la clase estándar (acceso frecuente) y, tras cumplir su tiempo de inactividad, son archivados para reducir costos.
- **Bucket `acme-archivado-historico`:** Funciona como la capa de respaldos backups. Al segregar los respaldos en un bucket independiente, se obtiene un mayor control de acceso y seguridad y se habilita la posibilidad de aplicar a futuro bloqueos de objetos (*Object Lock*) para evitar modificaciones accidentales o cumplir con regulaciones legales.

1.1.1. Políticas de Retención y Respaldo Automático

La gestión del ciclo de vida y la configuración de respaldos se realizaron mediante las siguientes directivas:

- **Transición del Almacenamiento Principal (`lifecycle.json`):** En el bucket operativo (`acme-datos-frecuentes`), se aplica una política que transiciona los archivos bajo el prefijo `logs/` a la clase de almacenamiento **S3 Glacier** a los 30 días de su creación, optimizando el costo del bucket principal de forma automatizada.
- **Estrategia de Backup Diario Incremental (`lifecycle_backup.json` y comando `aws s3 sync`):** Se automatiza el respaldo de los archivos sincronizándolos diariamente del bucket principal al de respaldo histórico mediante un script (`aws s3 sync`). Este comando aporta a la eficiencia de almacenamiento ya que solo copia los archivos nuevos o modificados, evitando redundancias. Para asegurar que el almacenamiento en el bucket de backup sea de mínimo costo, se le aplica una política de ciclo de vida que transiciona inmediatamente (`Days = 0`) cualquier archivo entrante a la clase Glacier.

1.2. Lección 2 – Bases de Datos Relacionales

Amazon RDS PostgreSQL (`acme-postgres-db`) es el servicio administrado elegido para estructurar los datos transaccionales core de la empresa (ventas, finanzas). Su naturaleza administrada permite delegar las tareas complejas de mantenimiento, aplicación de parches de seguridad y optimización del motor de base de datos a AWS, reduciendo la carga operativa del equipo de infraestructura.

¿Por qué PostgreSQL y no MariaDB, MySQL u Oracle? Oracle queda descartado al asumir que ACME no cuenta con una dependencia previa hacia ese motor en su entorno *on-premise*; incorporarlo implicaría costos de licenciamiento injustificados. PostgreSQL se prefiere sobre MySQL/MariaDB porque el caso de estudio involucra las áreas de **Finanzas** y **Ventas**, que requieren la máxima consistencia transaccional (*ACID*) y capacidad analítica para reportes complejos. Adicionalmente, su soporte nativo para el tipo **JSONB** ofrece una flexibilidad ideal para la integración con la estrategia NoSQL planteada en los requerimientos del proyecto.

1.2.1. Configuración AWS RDS con PostgreSQL

```
aws rds create-db-instance \
  --db-instance-identifier acme-postgres-db \
  --db-instance-class db.t3.micro \
  --engine postgres \
  --master-username jairoadmin \
  --master-user-password MiPasswordSeguro123 \
  --allocated-storage 20 \
  --backup-retention-period 7 \
  --multi-az \
  --endpoint-url=http://localhost:4566
```

Código 1.1: Configuración de instancia RDS con PostgreSQL

Multi-AZ: Activa la replicación sincrónica de los datos hacia una segunda **Zona de Disponibilidad (AZ)**, garantizando la tolerancia a fallos ante incidentes de infraestructura física en el centro de datos sin intervención manual.

Backup Retention Policy: Configurado a **7 días** para habilitar la recuperación a un punto específico en el tiempo (*Point-in-Time Recovery - PITR*), asegurando la continuidad del negocio ante escenarios de corrupción o pérdida accidental de datos.

1.2.2. Pruebas en SQLiteOnline

Para validar el diseño lógico de la base de datos relacional en este contexto académico, las pruebas de consultas se realizaron utilizando la plataforma **SQLiteOnline**. Esto permite simular el comportamiento del motor de base de datos (PostgreSQL) elegido para RDS y verificar la generación de reportes clave para el negocio de la empresa ACME.

Todo el código SQL desarrollado para la implementación de este esquema se encuentra documentado en el archivo `pruebas_postgres.sql`.

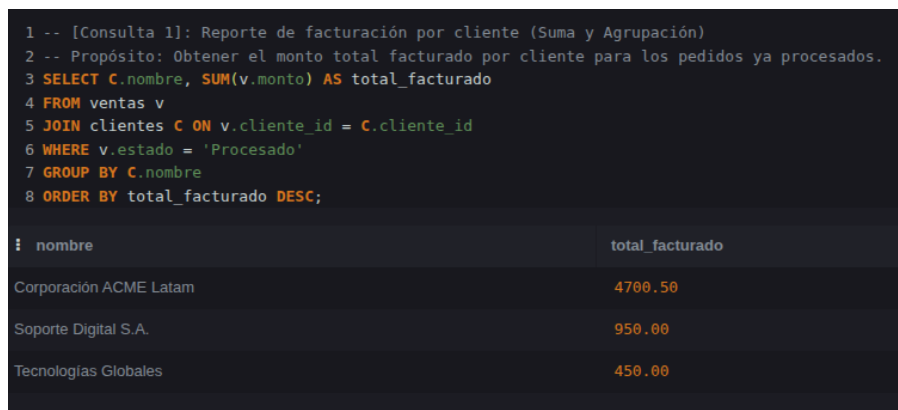
Paso 1: Configurar SQLiteOnline en modo PostgreSQL

1. Se ingresa a la plataforma web `sqliteonline.com`.
2. En la barra lateral izquierda, se selecciona la opción **PostgreSQL**.
3. Se inicia la conexión haciendo clic en **Click to connect** para inicializar un servidor PostgreSQL temporal.

Paso 2: Crear el esquema Se define la estructura relacional mediante la creación de la tabla principal `clientes` y la tabla transaccional `ventas`, la cual implementa una clave foránea para garantizar la integridad referencial. Posteriormente, se puebla la base de datos con un conjunto de datos iniciales que simulan operaciones comerciales estándar de la compañía.

Paso 3: Las Consultas de Reporte Con el entorno poblado, se ejecutan las siguientes tres consultas analíticas para responder a las necesidades de información de ACME:

1. **Consulta 1: Reporte de facturación por cliente (Suma y Agrupación)**
Calcula el acumulado total facturado de los pedidos procesados con éxito, agrupados por el nombre de cada cliente.



```
1 -- [Consulta 1]: Reporte de facturación por cliente (Suma y Agrupación)
2 -- Propósito: Obtener el monto total facturado por cliente para los pedidos ya procesados.
3 SELECT C.nombre, SUM(v.monto) AS total_facturado
4 FROM ventas v
5 JOIN clientes C ON v.cliente_id = C.cliente_id
6 WHERE v.estado = 'Procesado'
7 GROUP BY C.nombre
8 ORDER BY total_facturado DESC;
```

nombre	total_facturado
Corporación ACME Latam	4700.50
Soporte Digital S.A.	950.00
Tecnologías Globales	450.00

Figura 1.1: Resultado de la consulta de facturación agrupada por cliente.

2. **Consulta 2: Reporte de rendimiento por Región (Análisis comercial)**
Muestra la distribución geográfica de los ingresos operacionales, permitiendo evaluar el rendimiento comercial por zona.

```

1 -- [Consulta 2]: Reporte de rendimiento por Región (Análisis comercial)
2 -- Propósito: Analizar la distribución de ventas e ingresos totales según la región geográfica.
3 SELECT C.region, COUNT(v.venta_id) AS cantidad_ventas, SUM(v.monto) AS ingresos_totales
4 FROM ventas v
5 JOIN clientes C ON v.cliente_id = C.cliente_id
6 WHERE v.estado = 'Procesado'
7 GROUP BY C.region;
8

```

region	cantidad_ventas	ingresos_totales
Norteamérica	1	450.00
Sudamérica	3	5650.50

Figura 1.2: Resultado del análisis de ingresos totales distribuidos por región.

3. Consulta 3: Transacciones de alto valor (Filtro de auditoría)

Filtra de manera específica todas las ventas individuales que superan el umbral de \$1,000 USD para el control de transacciones críticas.

```

Run PostgreSQL
1 -- [Consulta 3]: Transacciones de alto valor (Filtro mayor a $1,000 USD)
2 -- Propósito: Identificar las ventas individuales de alta prioridad para auditorías comerciales.
3 SELECT v.venta_id, C.nombre, v.monto, v.fecha
4 FROM ventas v
5 JOIN clientes C ON v.cliente_id = C.cliente_id
6 WHERE v.monto > 1000.00
7 ORDER BY v.monto DESC;

```

venta_id	nombre	monto	fecha
3	Corporación ACME Latam	3200.00	2026-07-03
6	Tecnologías Globales	2100.00	2026-07-05
1	Corporación ACME Latam	1500.50	2026-07-01

Figura 1.3: Resultado del filtro de transacciones críticas mayores a \$1,000 USD.

1.3. Lección 3 – Base de Datos NoSQL

El departamento de **Soporte (TI)** de ACME gestiona un flujo constante e impredecible de incidencias reportadas por clientes y usuarios internos. Estas solicitudes tienen esquemas de datos variables (distintos campos según el tipo de incidente), ráfagas de escritura intensiva y necesidad de consultas dinámicas por múltiples atributos. Por estas razones, una base de datos relacional como PostgreSQL no es la solución óptima para este caso de uso; en su lugar, se utiliza una base de datos NoSQL.

Amazon DynamoDB (AcmeTicketsSoporte): Base de datos NoSQL de clave-valor seleccionada para el sistema de gestión de tickets del área de soporte técnico, debido a su capacidad de escalar horizontalmente de forma automática y ofrecer latencias de milisegundos de un solo dígito bajo cualquier volumen de carga.

Global Secondary Indexes (GSI): Se crearon 3 índices secundarios independientes (Index_Cliente, Index_Estado, Index_Prioridad) para superar la limitación de consulta por clave particionaria primaria, permitiendo búsquedas flexibles y eficientes por los atributos críticos del negocio sin penalizar el rendimiento ni el costo analítico.

1.3.1. Estrategia de Respaldo para DynamoDB

Para la tabla `AcmeTicketsSoporte`, se definió una estrategia de respaldo continuo para mitigar errores humanos y cumplir con requerimientos de continuidad operacional:

Recuperación en el tiempo (PITR): Activado de forma continua para permitir restauraciones con una precisión de segundos ante borrados o modificaciones accidentales de tickets, protegiendo la operación diaria del equipo de soporte. Si un agente elimina o altera por error un registro, PITR permite restaurar la tabla al estado exacto en que se encontraba en cualquier segundo dentro de los **35 días**.

```
# Habilitar Point-in-Time Recovery (PITR) para recuperación ante
↪ desastres de los últimos 35 días
aws dynamodb update-continuous-backups \
  --table-name AcmeTicketsSoporte \
  --point-in-time-recovery-specification
  ↪ PointInTimeRecoveryEnabled=true \
  --endpoint-url=http://localhost:4566
verificar "Habilitación de Backup Continuo (PITR) en DynamoDB"

# Verificación de que el backup continuo está activo en la tabla
aws dynamodb describe-continuous-backups --table-name
↪ AcmeTicketsSoporte --endpoint-url=http://localhost:4566 --query
↪ "ContinuousBackupsDescription.PointInTimeRecoveryDescription.Poi
↪ ntInTimeRecoveryStatus" --output text | grep -q
↪ "ENABLED"
verificar "Verificación de Backup Continuo (PITR) en estado ENABLED"
```

Código 1.2: Configuración para respaldo continuo con PITR

```
{
  "ContinuousBackupsDescription": {
    "ContinuousBackupsStatus": "ENABLED",
    "PointInTimeRecoveryDescription": {
      "PointInTimeRecoveryStatus": "ENABLED",
      "RecoveryPeriodInDays": 35
    }
  }
}
✓ [ÉXITO] Habilitación de Backup Continuo (PITR) en DynamoDB
✓ [ÉXITO] Verificación de Backup Continuo (PITR) en estado ENABLED
```

Figura 1.4: Verificación de estrategia de respaldo continuo en DynamoDB.

1.3.2. Integración funcional con una aplicación

Para validar el funcionamiento de la tabla y sus índices, se simuló el registro y consulta de un ticket de soporte utilizando AWS CLI como aplicación cliente. Esto demuestra el flujo completo de escritura y lectura sobre DynamoDB mediante el índice secundario `Index_Cliente`.


```

# Prueba de integración con una aplicación.
# Usamos como aplicación AWS CLI para mostrar que se acaba de
→ registrar un ticket:
aws dynamodb put-item \
  --table-name AcmeTicketsSoporte \
  --item '{
    "TicketId": {"S": "TK-1001"},
    "ClienteId": {"S": "CLIENTE-ACME-99"},
    "Estado": {"S": "Abierto"},
    "Prioridad": {"S": "Alta"},
    "FechaCreacion": {"S": "2026-07-02"},
    "Descripcion": {"S": "El servidor on-premise se quedo sin
→ espacio en disco duro"}
  }' \
  --endpoint-url=http://localhost:4566
verificar "Inserción de registro de prueba (put-item) en DynamoDB"

# Comprobamos que se registró correctamente el ticket:
aws dynamodb query \
  --table-name AcmeTicketsSoporte \
  --index-name Index_Cliente \
  --key-condition-expression "ClienteId = :v1" \
  --expression-attribute-values '{":v1": {"S": "CLIENTE-ACME-99"}}' \
  --endpoint-url=http://localhost:4566
verificar "Consulta de verificación (query) en DynamoDB"

```

Código 1.3: Insertar y consulta en Amazon DynamoDB

```

✓ [ÉXITO] Inserción de registro de prueba (put-item) en DynamoDB
{
  "Items": [
    {
      "TicketId": {
        "S": "TK-1001"
      },
      "ClienteId": {
        "S": "CLIENTE-ACME-99"
      },
      "Estado": {
        "S": "Abierto"
      },
      "Prioridad": {
        "S": "Alta"
      },
      "FechaCreacion": {
        "S": "2026-07-02"
      },
      "Descripcion": {
        "S": "El servidor on-premise se quedo sin espacio en disco duro"
      }
    }
  ],
  "Count": 1,
  "ScannedCount": 1,
  "ConsumedCapacity": null
}
✓ [ÉXITO] Consulta de verificación (query) en DynamoDB

```

Figura 1.5: Resultado de la inserción y consulta en DynamoDB.

1.4. Lección 4 – Servicios de Cómputo

AWS Lambda (*AcmeProcesarVentas*): Función serverless seleccionada para ejecutar la lógica de procesamiento de reportes de ventas bajo demanda. Al ser un servicio administrado, elimina la necesidad de aprovisionar o mantener servidores (como EC2), facturando únicamente por el tiempo de cómputo consumido y escalando de forma automática y proporcional al volumen de solicitudes entrantes.

1.4.1. Prueba de Escalabilidad bajo Carga

El enunciado solicita demostrar la capacidad de escalado elástico del servicio de cómputo. Para ello, se implementó una función que simula una carga computacional intensiva en CPU, representando el procesamiento real de un reporte de ventas:

```

import json
import time

def lambda_handler(event, context):
    # Simula un procesamiento pesado en el procesador (CPU)
    inicio = time.time()
    contador = 0
    for i in range(1000000):
        contador += i
    fin = time.time()

    return {
        'statusCode': 200,
        'body': json.dumps({
            'mensaje': 'Reporte de ventas ACME procesado con exito',
            'tiempo_procesamiento_segundos': fin - inicio
        })
    }

```

Código 1.4: Función Lambda que simula el procesamiento intensivo de un reporte de ventas.

Sobre esta función se ejecutaron **10 invocaciones consecutivas** para simular una ráfaga de peticiones simultáneas al sistema de ventas:

```

# Realizar Pruebas de Escalabilidad y Ejecución bajo Carga
echo "Ejecutando pruebas de escalabilidad sobre Lambda..."
for i in {1..10}; do
    aws lambda invoke --function-name AcmeProcesarVentas
    ↪ --endpoint-url=http://localhost:4566
    ↪ resultados-prueba-carga/response_${i}.json >/dev/null
    verificar "Invocación $i de Lambda"
done

```

Código 1.5: Script de prueba de escalabilidad: 10 invocaciones consecutivas sobre AcmeProcesarVentas.

Los resultados individuales de cada invocación se encuentran en los archivos **resultados-prueba-carga-response_*.json**, disponibles en el repositorio de GitHub. Los tiempos registrados evidencian un comportamiento estable y predecible bajo carga, con un **promedio de 0.060 s** y una desviación estándar de **0.0011 s** entre invocaciones. Esto confirma que Lambda escala de forma elástica y aislada ante ráfagas de peticiones, sumando una garantía de alta disponibilidad del procesamiento de ventas sin necesidad de infraestructura física dedicada.

1.5. Lección 5 – Servicios de Red en la Nube

Aquí se detallan los servicios de red usados, responsables de segmentar el tráfico, aislar los recursos críticos y controlar el acceso entre las distintas capas de la arquitectura de ACME.

- **Amazon VPC (10.0.0.0/16)**: Red virtual lógicamente aislada creada mediante `create-vpc` que actúa como el perímetro de seguridad general para delimitar la infraestructura de ACME en la nube.
- **Subredes Públicas A (10.0.1.0/24) y B (10.0.3.0/24)**: Segmentos de red distribuidos en distintas zonas de disponibilidad (`us-east-1a` y `us-east-1b`) diseñados para alojar recursos que requieren acceso directo desde internet, garantizando alta disponibilidad geográfica.
- **Subred Privada A (10.0.2.0/24)**: Zona de red aislada orientada a proteger la base de datos relacional y los recursos críticos de procesamiento que no deben exponerse al internet público bajo ninguna circunstancia.
- **Grupo de Seguridad del ALB (acme-balanc-sg)**: Firewall a nivel de recurso que restringe el acceso al balanceador, permitiendo la entrada de tráfico web únicamente por los puertos estándar HTTP (80) y HTTPS (443) desde cualquier origen (0.0.0.0/0).
- **Grupo de Seguridad de la Base de Datos (acme-rds-sg)**: Capa de protección perimetral interna que bloquea todo el tráfico hacia el puerto predeterminado de PostgreSQL (5432), autorizando de forma estricta únicamente las peticiones originadas dentro del rango CIDR corporativo (10.0.0.0/16).
- **Asociación RDS y Security Group (modify-db-instance)**: Operación para vincular el firewall interno a la base de datos `acme-postgres-db`. *Nota para el informe*: Debido a limitaciones operativas del simulador Floci Community, esta asociación de grupos no persiste en el entorno local; sin embargo, en un entorno productivo de AWS real, confina la base de datos al tráfico autorizado.
- **Inyección de Lambda en VPC (update-function-configuration)**: Comando para dotar a la función `AcmeProcesarVentas` de interfaces de red virtuales (ENI) dentro de la subred privada `SUBNET_PRIV_A_ID`. *Nota para el informe*: Floci acepta el comando pero no persiste la configuración `VpcConfig`. En AWS real, este mecanismo es el que le permite a la Lambda consumir datos de RDS de forma segura y privada, sin enrutamiento externo.
- **Application Load Balancer (acme-web-alb)**: Balanceador de carga de capa 7 encargado de recibir el tráfico web externo y distribuirlo hacia el backend. Se acopla de forma exclusiva a las subredes públicas A y B para mantener la separación de funciones frente a la zona privada.

1.6. Lección 6 – Notificación y Mensajería

Amazon SQS (*acme-cola-eventos*): Cola de mensajería asíncrona implementada para desacoplar el flujo de datos. Dentro de la arquitectura de ACME, recibe las solicitudes provenientes del balanceador de carga y las deriva de forma controlada hacia la función Lambda *AcmeProcesarVentas* (configurada en la Lección 4) para el procesamiento de reportes de ventas, protegiendo al backend de sobrecargas.

SQS Dead-Letter Queue (*acme-cola-eventos-dlq*): Cola de aislamiento dedicada a almacenar los mensajes que no pudieron ser procesados con éxito tras superar el límite de reintentos configurado.

Amazon SNS (*acme-alertas-operacionales*): Tópico de mensajería *Pub/Sub* configurado para centralizar el enrutamiento y envío de notificaciones críticas del sistema. Este tópico será el canal receptor de las alarmas operacionales definidas en la Lección 8 (CloudWatch), desde donde se despacharán las alertas al equipo de soporte.

Suscripción de Email (*alertas@acme-empresa.com*): Único suscriptor registrado al tópico SNS, actuando como el canal oficial de comunicación para la recepción de alertas operacionales inmediatas. Su integración con CloudWatch se detalla en la Lección 8.

1.6.1. Política de reintentos

Regla de resiliencia aplicada mediante `set-queue-attributes` que extrae automáticamente un mensaje de la cola principal y lo mueve a la DLQ si este falla un total de 3 veces consecutivas (`maxReceiveCount: 3`), evitando atascos en la línea de procesamiento.

```
# Configurar la política de reintentos (redrive policy)
aws sqs set-queue-attributes \
  --queue-url $COLA_URL \
  --attributes "{\"RedrivePolicy\": \"{\\\"deadLetterTargetArn\\\":\\\"_
  ↪ \\\"$COLA_DLQ_ARN\\\",\\\"maxReceiveCount\\\":\\\"3\\\"}\"\",
  ↪ \\\"VisibilityTimeout\\\": \\\"30\\\", \\\"MessageRetentionPeriod\\\":
  ↪ \\\"86400\\\"}\"\" \
  --endpoint-url=http://localhost:4566
verificar "Configurar política de Dead-Letter Queue (DLQ) en SQS"
```

Código 1.6: Configuración de la política de reintentos y Dead-Letter Queue en SQS.

1.7. Lección 7 – Servicios Simples de Alojamiento Web y Contenidos

Para desplegar el sitio web corporativo de ACME, se implementó una arquitectura orientada a la alta velocidad de carga y mínimo costo operativo, combinando el almacenamiento de objetos con una red de entrega de contenido:

- **Amazon S3 Static Website Hosting** (*acme-sitio-web-frontend*): Bucket con-

figurado exclusivamente para almacenar y servir el código estático de la web (HTML, CSS y JavaScript), eliminando por completo los costos de procesamiento, aprovisionamiento y mantenimiento de servidores web tradicionales.

- **Amazon CloudFront (CDN):** Red de distribución de contenido que actúa como origen perimetral frente al bucket de S3. Almacena en caché los archivos del sitio en ubicaciones de borde cercanas a los usuarios, optimizando drásticamente los tiempos de carga globales y protegiendo el bucket de accesos directos desde internet.

1.7.1. Configuraciones de seguridad para el servicio web

Para asegurar la entrega de contenido y proteger los activos del sitio web de ACME, se implementaron dos medidas de seguridad fundamentales:

1. **Certificado SSL/TLS (ACM):** Se solicitó un certificado digital a través de AWS Certificate Manager (ACM) para el dominio principal (**acme-empresa.com**). Este es el paso esencial para habilitar conexiones seguras bajo el protocolo HTTPS en CloudFront, garantizando que el tráfico entre los usuarios y la red perimetral viaje cifrado.
2. **Cifrado en Reposo (SSE-S3):** Se habilitó el cifrado del lado del servidor mediante el algoritmo AES-256 en el bucket **acme-sitio-web-frontend**. De esta forma, todos los archivos del sitio se almacenan de forma segura, protegiendo la información ante cualquier intento de acceso no autorizado directo al almacenamiento físico.

A continuación, se presentan los comandos utilizados en el script para aplicar ambas configuraciones de seguridad:

```

# SEGURIDAD 1 (SSL): Solicitar certificado TLS/SSL en ACM para el
↳ dominio de la web corporativa
# En AWS real, este certificado se asocia a la distribución
↳ CloudFront para forzar HTTPS en todo el sitio.
CERT_ARN=$(aws acm request-certificate \
  --domain-name acme-empresa.com \
  --validation-method DNS \
  --query 'CertificateArn' --output text \
  --endpoint-url=http://localhost:4566)
verificar "Solicitud de certificado SSL/TLS (ACM) para
↳ acme-empresa.com"
echo " → Certificado ARN: $CERT_ARN"
echo " → En AWS real se asociaría a CloudFront para redirigir HTTP
↳ → HTTPS (ViewerProtocolPolicy: redirect-to-https)"

# SEGURIDAD 2 (Permisos): Habilitar cifrado del lado del servidor
↳ (SSE-S3 / AES-256) en el bucket web
# Todos los objetos del sitio quedan cifrados en reposo; protege el
↳ contenido ante acceso no autorizado al almacenamiento.
aws s3api put-bucket-encryption \
  --bucket acme-sitio-web-frontend \
  --server-side-encryption-configuration '{"Rules":[{"ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"AES256"}}]}'
↳ \
  --endpoint-url=http://localhost:4566
verificar "Cifrado SSE-S3 (AES-256) habilitado en bucket
↳ acme-sitio-web-frontend"

```

Código 1.7: Comandos para la seguridad del servicio web

1.8. Lección 8 – Monitoreo y Correlación de Incidentes

En esta lección se implementa el monitoreo activo del sistema mediante Amazon CloudWatch. Esto se conecta con lo realizado en la Lección 6, donde configuramos la cola SQS y el canal de correo en SNS. Ahora, al vigilar las métricas de la Lambda y de la cola de mensajería, podemos activar alarmas que utilicen ese canal de SNS ya creado para enviar alertas automáticas al equipo de soporte cuando ocurra un incidente.

1. Telemetría y Métricas Clave Monitorizadas

Se inyectaron datos de prueba (put-metric-data) para establecer la línea base de tres indicadores críticos de rendimiento y disponibilidad en la arquitectura:

- **Métrica 1 - Invocaciones de Lambda (Invocations):** Recopilada bajo el espacio de nombres nativo AWS/Lambda y dimensionada específicamente para la función AcmeProcesarVentas. Permite auditar el volumen exacto de ejecuciones, la tasa de

uso del cómputo y la demanda transaccional real del sistema de ventas.

- **Métrica 2 - Errores de Lambda (Errors):** Registrada bajo el namespace `AWS/Lambda` para mapear de forma continua las excepciones del código, fallos de sintaxis o problemas de conectividad interna experimentados por la función `AcmeProcesarVentas`.
- **Métrica 3 - Mensajes Pendientes SQS (`ApproximateNumberOfMessagesVisible`):** Capturada en el espacio de nombres `AWS/SQS` para la cola `acme-cola-eventos`. Mide la cantidad exacta de mensajes acumulados que están listos para ser procesados, sirviendo como el sensor principal para identificar cuellos de botella en el backend.

2. Configuración Analítica de Alarmas con Umbrales

Para automatizar la detección de incidentes, se implementaron dos alarmas con lógica matemática y periodos de evaluación estrictos mediante `put-metric-alarm`:

- **Alarma 1 - Errores Críticos de Aplicación (`acme-alarma-errores-criticos`):** Monitorea la métrica `Errors` sumando su comportamiento (`Statistic Sum`) durante un bloque continuo de 5 minutos (`Period 300`). Si el contador es igual o superior a 5 fallos (`Threshold 5`), la alarma cambia inmediatamente a estado crítico.
- **Alarma 2 - Saturación de Mensajería (`acme-alarma-cola-saturada`):** Monitorea la métrica de mensajes visibles en SQS sumando los elementos en cola en un periodo de 5 minutos. Si se acumulan 100 o más mensajes pendientes sin procesar (`Threshold 100`), se interpreta que el cómputo de la Lambda está superado o que hay lentitud en la base de datos relacional, disparando el estado de alerta.

4. Gobernanza y Retención Automática de Logs

- **Grupo y Flujo de Logs (`/aws/lambda/AcmeProcesarVentas`):** Se valida de forma controlada las estructuras de logs históricos para la Lambda.
- **Política de Ciclo de Vida (`put-retention-policy`):** Se configuró una acción automática que fuerza una retención estricta de logs de 30 días (`--retention-in-days 30`). Esto garantiza que el sistema limpie de forma autónoma el almacenamiento de bitácoras redundantes y evite sobrecostos imprevistos, manteniendo solo la ventana de tiempo necesaria para auditorías de soporte técnico.

1.8.1. Acción Automática (Notificación a SNS)

- Ambas alarmas tienen vinculada la propiedad `--alarm-actions` apuntando directamente a la variable dinámica `\$TOPIC_ARN` (`acme-alertas-operacionales`).
- **Mecanismo de acción:** En el instante exacto en que CloudWatch detecta que cualquiera de las dos métricas cruza el umbral crítico definido, el servicio cambia su estado de salud a `ALARM` e inyecta de manera automática la bitácora de error en el “megáfono” de SNS.

- A través de este patrón de integración nativo, SNS propaga el incidente de inmediato a la dirección de correo configurada (`alertas@acme-empresa.com`), notificando al equipo de administración en tiempo real con todos los metadatos y dimensiones del fallo.

En el siguiente código se muestran los comandos bash para la configuración de alarma.

```
# ALARMA 1: Monitorea errores de Lambda
aws cloudwatch put-metric-alarm \
  --alarm-name "acme-alarma-errores-criticos" \
  --alarm-description "Alarma: Se detectaron errores críticos en la
↳ función Lambda de ACME" \
  --namespace "AWS/Lambda" \
  --metric-name "Errors" \
  --dimensions Name=FunctionName,Value=AcmeProcesarVentas \
  --statistic Sum \
  --period 300 \
  --evaluation-periods 1 \
  --threshold 5 \
  --comparison-operator GreaterThanOrEqualToThreshold \
  --alarm-actions $TOPIC_ARN \
  --endpoint-url=http://localhost:4566
verificar "Creación de Alarma CloudWatch 1 (Errores Lambda)"

# ALARMA 2: Monitorea mensajes SQS
aws cloudwatch put-metric-alarm \
  --alarm-name "acme-alarma-cola-saturada" \
  --alarm-description "Alarma: La cola SQS de ACME tiene demasiados
↳ mensajes pendientes" \
  --namespace "AWS/SQS" \
  --metric-name "ApproximateNumberOfMessagesVisible" \
  --dimensions Name=QueueName,Value=acme-cola-eventos \
  --statistic Sum \
  --period 300 \
  --evaluation-periods 1 \
  --threshold 100 \
  --comparison-operator GreaterThanOrEqualToThreshold \
  --alarm-actions $TOPIC_ARN \
  --endpoint-url=http://localhost:4566
verificar "Creación de Alarma CloudWatch 2 (Saturación SQS)"
```

Código 1.8: Comandos de creación de alarmas en CloudWatch para Lambda y SQS.

2. Diagrama de Arquitectura

Se presentan tres diagramas que en conjunto describen la arquitectura de ACME: un diagrama general con el flujo principal de acceso, red y datos; un diagrama de almacenamiento; y un diagrama de mensajería y monitoreo.

El primer diagrama, el más general, muestra el flujo de una solicitud que viene desde internet. Los usuarios acceden por dos vías:

La primera vía se accede hacia el contenido estático del sitio web, servido a través de una distribución de **CloudFront** que apunta al bucket **acme-sitio-web-frontend**, configurado como hosting web estático.

La segunda vía ingresa a la VPC (10.0.0.0/16) a través del Application Load Balancer **acme-web-alb**, desplegado en dos subredes públicas en zonas de disponibilidad distintas. Desde el ALB, las solicitudes se depositan en la cola **acme-cola-eventos** (SQS), la cual cuenta con una Dead Letter Queue asociada para los mensajes que agotan sus reintentos. Los mensajes de la cola son consumidos por la función **AcmeProcesarVentas** (Lambda, Python 3.9), desplegada en la subred privada y conectada a esta mediante una interfaz de red elástica (ENI). Desde ahí, la función Lambda se comunica con la base de datos relacional **acme-postgres-db** (RDS PostgreSQL) dentro de la misma subred privada, y con la tabla **AcmeTicketsSoporte** en DynamoDB, un servicio gestionado fuera de la VPC.

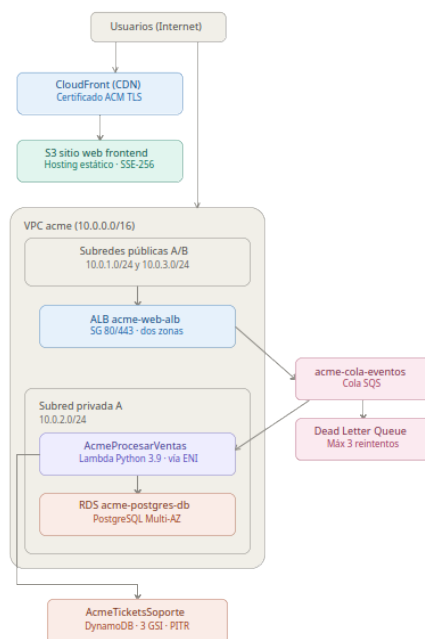


Figura 2.1: Diagrama general de la arquitectura ACME

El segundo diagrama corresponde al almacenamiento. El bucket operativo **acme-datos-frecuentes** recibe una sincronización incremental diaria hacia el bucket de respaldo **acme-archivado-historico**. Cada bucket cuenta con su propia política de ciclo de vida: el bucket operativo transi-

ciona sus archivos desde almacenamiento standard a Glacier tras 30 días de inactividad, mientras que el bucket de respaldo transiciona de forma inmediata cualquier archivo que ingresa a él.

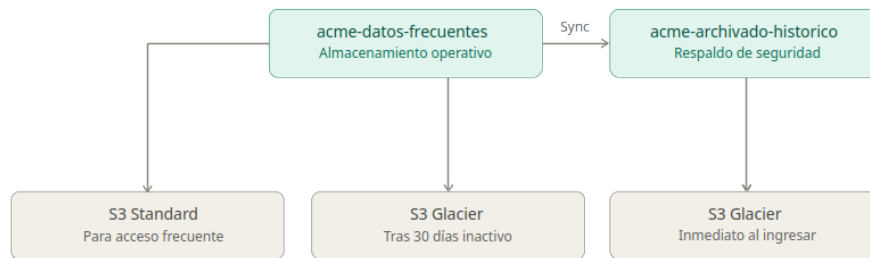


Figura 2.2: Diagrama del Almacenamiento.

El tercer diagrama corresponde a las alarmas y notificaciones. El servicio **CloudWatch** supervisa tanto a la función Lambda como a la cola SQS: en el caso de la Lambda, mediante el grupo de logs `/aws/lambda/AcmeProcesarVentas`, con una retención de 30 días; en el caso de SQS, mediante métricas del número de mensajes visibles en la cola. Dos alarmas monitorean estos servicios: **acme-alarma-errores-criticos**, que se activa si la Lambda registra 5 o más errores en 5 minutos, y **acme-alarma-cola-saturada**, que se activa si la cola SQS acumula 100 o más mensajes visibles en el mismo período. Ambas alarmas notifican al tema **acme-alertas-operacionales** (SNS), el cual tiene como suscriptor la dirección de correo `alertas@acme-empresa.com`.

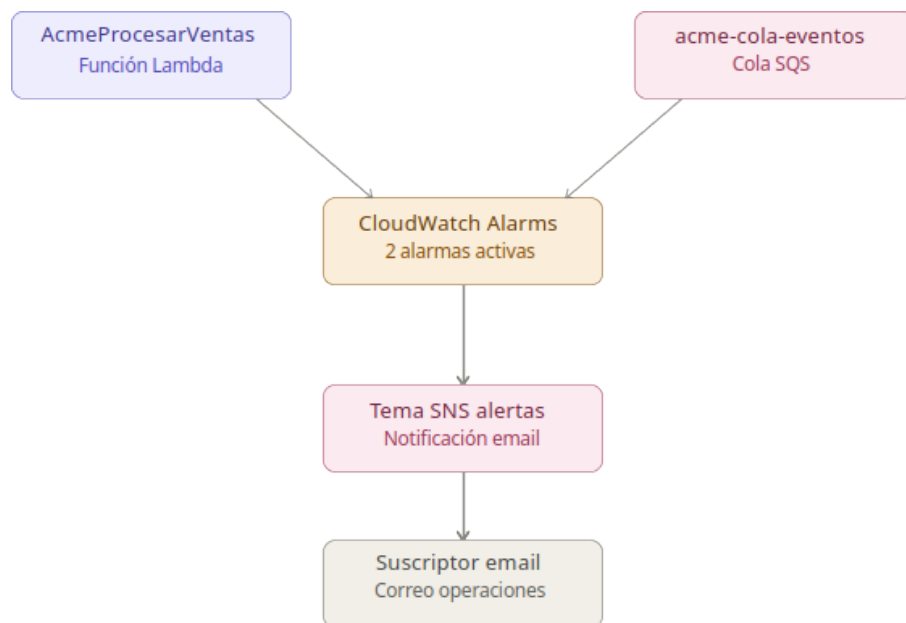


Figura 2.3: Diagrama de alarmas y notificaciones de la arquitectura.

3. Conclusión

Al completar todas las lecciones del enunciado, se consolidó un sistema en la nube que reduce los costos al no depender de infraestructura física *on-premise*, asegura la escalabilidad y alta disponibilidad en almacenamiento y procesamiento, protege el tráfico mediante redes seguras y vigila su propia salud con alarmas automáticas. En definitiva, este portafolio demuestra de forma práctica cómo aplicar con éxito los conceptos del bootcamp en un escenario empresarial real.

Índice de figuras

1.1.	Resultado de la consulta de facturación agrupada por cliente.	3
1.2.	Resultado del análisis de ingresos totales distribuidos por región.	4
1.3.	Resultado del filtro de transacciones críticas mayores a \$1,000 USD. . . .	4
1.4.	Verificación de estrategia de respaldo continuo en DynamoDB.	5
1.5.	Resultado de la inserción y consulta en DynamoDB.	7
2.1.	Diagrama general de la arquitectura ACME	15
2.2.	Diagrama del Almacenamiento.	16
2.3.	Diagrama de alarmas y notificaciones de la arquitectura.	17

Índice de Códigos

1.1.	Configuración de instancia RDS con PostgreSQL	2
1.2.	Configuración para respaldo continuo con PITR	5
1.3.	Insertar y consulta en Amazon DynamoDB	6
1.4.	Función Lambda que simula el procesamiento intensivo de un reporte de ventas.	8
1.5.	Script de prueba de escalabilidad: 10 invocaciones consecutivas sobre AcmeProcesarVentas.	8
1.6.	Configuración de la política de reintentos y Dead-Letter Queue en SQS. .	10
1.7.	Comandos para la seguridad del servicio web	12
1.8.	Comandos de creación de alarmas en CloudWatch para Lambda y SQS. .	14